AF 52

Synopsis
DripGuard: Smart IV Infusion Monitoring & Prediction System

Submitted in partial fulfilment of the requirements

of the degree of

Second Year of Engineering
In
Computer Science and Engineering
(Data Science)

by

Khushi Patil [EU1258042]

Mrunali Kamble [EU1258041]

Swati Mahto [EU1258046]

Under the Guidance of

Prof. Sachin Sase

Assistant Professor, CSE(DS) Department , SJCEM

DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING (DATA SCIENCE)
ST. JOHN COLLEGE OF ENGINEERING & MANAGEMENT
2026-2027

St. John College of Engineering and Management
Autonomous Institute
(A Christian Religious Minority Institution)
Approved by AICTE and DTE, Affiliated to University of Mumbai / MSBTE
DTE Code : 3218 AICTE Permanent ID : 1-4790201
NAAC Accredited with Grade 'A+', Three Programs NBA Accredited

Certificate of Approval

This is to certify that Khushi Patil, Mrunali Kamble, Swati Mahto are bonafide students of S.Tech in Computer Science and Engineering (Data Science)-Department, St. John College of Engineering and Management, Mumbai. They have satisfactorily completed the requirements of Synopsis as prescribed by **ST. JOHN COLLEGE OF ENGINEERING AND MANAGEMENT (An Autonomous College affiliated to University of Mumbai),** while working on "DripGuard: Smart IV Infusion Monitoring & Prediction System".

Signature	:		Signature	:	
Name	:	Prof. Sachin Sase	Name	:	Mrs. Joslyn Gracias e Vaz
Designation:		Assistant Professor	Designation:		HOD- CSE(DS) Dept.

Signature: ----------

Name : Dr. Kamal Shah
Principal,
St. John College of Engineering and Management.

Declaration

I/we declare that this written submission represents my/our ideas in my/our own words and where others ideas or words have been included, I/we have adequately cited and referenced the original sources. I/we also declare that I/we have adhered to all principles of academic honesty and integrity and have not misrepresented or fabricated or falsified any idea/data/fact/source in my/our submission. I/we understand that any violation of the above will be cause for disciplinary action by the Institute and can also evoke penal action from the sources which have thus not been properly cited or from whom proper permission has not been taken when needed.

(Signature)

Khushi Patil, Mrunali Kamble, Swati Mahto
(Student Name)

Date: 23/10/2026

Acknowledgement

We express our sincere gratitude to all those who have directly and indirectly contributed to the successful preparation of this project synopsis.

We are deeply grateful to **Dr. Kamal Shah, Principal, St. John College of Engineering and Management**, for providing an excellent academic environment and the necessary facilities to undertake this project.

We extend our heartfelt thanks to the **Mrs. Joslyn Gracias**, **Head of the Department of Computer Science and Engineering (Data Science)** for their constant encouragement, valuable suggestions, and continuous support throughout the preparation of this synopsis.

We would like to express our sincere appreciation to our project guide, **Prof. Sachin Sase**, for their invaluable guidance, constructive suggestions, technical expertise, and continuous motivation. Their encouragement and insightful feedback have been instrumental in shaping this project synopsis.

We also acknowledge all the faculty members and laboratory staff of the Department of Computer Science and Engineering (Data Science) for their timely assistance, cooperation, and technical support.

Finally, we express our heartfelt gratitude to our parents, family members, friends, and classmates for their constant encouragement, moral support, and cooperation throughout this work.

Group Members

1. Khushi Patil
2. Mrunali Kamble
3. Swati Mahto

ABSTRACT

Intravenous (IV) infusion is commonly used in healthcare to deliver fluids and medications to patients. Manual monitoring of an infusion can require repeated observation by healthcare staff, and delays in identifying a stopped or unusually slow drip may affect timely intervention. The proposed project, DripGuard: Smart IV Infusion Monitoring and Prediction System, aims to develop a low-cost prototype that monitors IV drip activity and provides alerts when abnormal conditions are detected.

The proposed system uses an IR sensor to detect drops in an IV drip chamber. A NodeMCU ESP8266 microcontroller processes the sensor readings and controls visual indicators and an audible buzzer according to the programmed monitoring conditions. The system is designed to estimate flow rate, delivered volume, remaining volume, and approximate remaining infusion time using configured parameters. A Python-based dashboard and database are proposed to display and store monitoring information received from the device over Wi-Fi.

The prototype is intended to demonstrate sensor-based monitoring, local alerts, and software-based observation of infusion activity. Its performance will depend on sensor calibration, the configured drop factor, the accuracy of drop detection, and the reliability of communication between the hardware and software. The project is an academic prototype and is not intended to replace clinical supervision or a certified medical infusion monitoring device.

Keywords: Intravenous infusion, Internet of Things (IoT), NodeMCU ESP8266, IR sensor, drip monitoring, alert system, Python dashboard.

C O N T E N T S

- List of Figures i
- List of Tables i.
- Abbreviations and symbols ii.

Chapter No.	Topic	Page No.
Chapter 1	Introduction: Background, Problem Statement, Need, Objectives, Scope, Expected Outcomes	**1-3**
Chapter 2	Literature Review: Survey table, Research Gap, Novelty/Innovation	**4-8**
Chapter 3	Proposed Methodology: Working Principle, Architecture/Block Diagram, Flowchart, Methodology, Components	**9-13**
Chapter 4	Project Planning: WBS, Gantt Chart, Roles, Risk Assessment	**14-16**
Chapter 5	Resources & Feasibility: Technical/Economic/Operational Feasibility, Budget, BOM	**17-19**
Chapter 6	Expected Results & Impact: Expected outcomes, Industrial relevance, Sustainability, SDG mapping, Future Scope	**20-23**
Chapter 7	Conclusion	**24**
References	IEEE style; 10–20 references	**25**
Appendix	CAD drawings, calculations, questionnaires, etc. (optional)	**26-28**

List of Figures

Figure No.	Figure Name	Page No.
Figure 3.1	Block Diagram of the DripGuard System	10
Figure 3.2	Flowchart of the Drip Monitoring and Alert Process	11
Figure 4.1	Project Gantt Chart	14
Figure A.1	Circuit Diagram of the DripGuard System	26
Figure A.2	Actual Hardware Prototype of the DripGuard System	28

List of Tables

Table No.	Table Name	Page No.
Table 2.1	Literature Review of Existing IV Infusion Monitoring Systems	4-6
Table 3.1	Components and Materials Required	12-13
Table 4.2	Roles and Responsibilities	15
Table 4.3	Risk Assessment	15-16
Table 5.1	Estimated Project Budget	18
Table 5.2	Bill of Materials (BOM)	18-19

Abbreviations and symbols

- **IV**: Intravenous
- **IoT**: Internet of Things
- **IR**: Infrared
- **MCU**: Microcontroller Unit
- **ESP8266**: Wi-Fi-enabled microcontroller system-on-chip
- **Wi-Fi**: Wireless networking technology
- **GPIO**: General-Purpose Input/Output
- **LED**: Light-Emitting Diode
- **USB**: Universal Serial Bus
- **HTTP**: Hypertext Transfer Protocol
- **DB**: Database
- **SQLite**: Lightweight relational database engine
- **WBS**: Work Breakdown Structure
- **BOM**: Bill of Materials
- **SDG**: Sustainable Development Goal
- **mL**: Millilitre
- **mL/h**: Millilitres per hour
- **drops/min**: Drops per minute
- **drops/mL**: Drops per millilitre
- **V**: Volt
- **Ω**: Ohm

CHAPTER 1: INTRODUCTION

1.1 Background

Intravenous (IV) infusion is a commonly used medical procedure in which fluids, medications, or nutrients are delivered directly into a patient's bloodstream through an infusion set. Maintaining an appropriate infusion rate is important for ensuring that the prescribed fluid is administered as intended. In conventional gravity-based IV infusion systems, the flow rate is controlled manually using a roller clamp, and healthcare staff periodically observe the drip chamber to monitor the flow of fluid.

Manual monitoring can become challenging in busy healthcare environments, where staff must attend to multiple patients simultaneously. Changes in the drip rate, interruption of fluid flow, or an emptying IV bag may not be noticed immediately if continuous observation is not possible. These situations highlight the need for a system that can assist with monitoring the IV infusion process and provide timely alerts when abnormal conditions are detected.

DripGuard: Smart IV Infusion Monitoring and Prediction System is proposed as an IoT-based prototype to monitor IV fluid drops using an infrared (IR) sensor and a NodeMCU ESP8266 microcontroller. The sensor detects drops passing through the IV drip chamber, while the microcontroller processes the readings to estimate the drip rate, delivered fluid volume, remaining fluid volume, and estimated remaining infusion time based on configured parameters.

The system uses visual indicators and a buzzer to communicate the detected status. It also sends monitoring data through Wi-Fi to a Python-based dashboard, where readings can be displayed and stored in an SQLite database. The proposed system aims to support easier observation of IV infusion conditions and demonstrate how low-cost embedded hardware and software can be integrated for remote monitoring. It is an academic prototype and is not intended to replace clinical supervision or certified medical equipment.

1.2 Problem Statement

Conventional gravity-based IV infusion systems require healthcare staff to monitor the drip chamber and manually adjust the flow rate. In environments where multiple patients require attention, continuous observation of every infusion may be difficult. Delayed detection of interrupted flow, a very low drip rate, or a nearly depleted IV fluid bag may lead to delays in responding to an infusion-related issue.

Many basic IV setups do not provide automated drop counting, estimated remaining fluid volume, or a centralized digital display of infusion status. This creates a need for an affordable monitoring prototype that can detect IV drops, process the readings, display the estimated infusion status, and generate alerts when the monitored flow falls outside the configured operating condition.

Therefore, the problem addressed by DripGuard is the development of an IoT-based prototype that monitors IV drip activity, estimates infusion-related parameters, and provides local and dashboard-based status information to assist observation. The system is intended as a supplementary monitoring aid, not as an autonomous infusion controller or a substitute for trained medical personnel.

1.3 Need of the Project

The need for DripGuard arises from the importance of monitoring gravity-based IV infusion and identifying changes in drip activity in a timely manner. Manual monitoring depends on periodic observation by healthcare staff, and this can be challenging when several patients require attention.

The proposed system addresses this need through the following features:

1. Automated drop detection: An IR sensor detects drops passing through the IV drip chamber, reducing the need for continuous manual drop counting.
2. Infusion parameter estimation: The NodeMCU processes drop counts to estimate the drip rate, delivered volume, remaining volume, and remaining infusion time using configured values.
3. Immediate local indication: LEDs indicate the monitored status, while a buzzer provides an audible alert under the configured critical condition.
4. Wireless monitoring: Wi-Fi communication transfers readings to a Python-based dashboard for viewing infusion information.
5. Data recording: An SQLite database stores monitoring readings for later inspection and analysis.
6. Affordable prototyping: The use of commonly available components provides a practical platform for demonstrating embedded systems, IoT communication, sensor interfacing, and basic data analysis.

The project provides an opportunity to explore how an integrated hardware and software system can support IV infusion observation. Any use in a clinical environment would require appropriate validation, safety assessment, and regulatory approval.

1.4 Objectives

The main objective of DripGuard is to design and develop an IoT-based prototype for monitoring IV infusion drops and estimating infusion-related parameters.

The specific objectives are:
1. To interface an IR sensor with the NodeMCU ESP8266 for detecting drops in an IV drip chamber.
2. To count detected drops and calculate the drip rate over a defined time interval.
3. To estimate delivered fluid volume, remaining fluid volume, and remaining infusion time using configurable bag-volume and drip-factor values.
4. To classify the monitored condition into normal, attention, and critical statuses according to the programmed rules.
5. To provide local status indication through green, yellow, and red LEDs and an audible buzzer.
6. To establish Wi-Fi communication between the NodeMCU and a Python-based monitoring dashboard.
7. To display infusion readings and store monitoring data in an SQLite database.
8. To evaluate the prototype's functional operation under controlled demonstration conditions and identify its limitations and possible improvements.

1.5 Scope

The scope of DripGuard covers the development and demonstration of a low-cost prototype for monitoring a gravity-based IV infusion setup. The project integrates an IR sensor, NodeMCU

ESP8266, visual and audible indicators, wireless communication, a Python-based dashboard, and an SQLite database.

The prototype is intended to perform the following functions:

- Detect drops passing through the IV drip chamber and count them over a defined interval.
- Estimate the drip rate, delivered volume, remaining fluid volume, and remaining infusion time from the drop count and configured parameters.
- Indicate normal, attention, or critical status through LEDs and activate the buzzer when the programmed critical condition is detected.
- Transmit sensor readings and status information to a dashboard through Wi-Fi.
- Display and store readings to support monitoring and review.
- Demonstrate a basic multi-bed dashboard arrangement in which B01 represents the physical hardware connection and additional beds may be simulated in software.

The project is limited to monitoring and estimation. It does not physically regulate the IV flow, administer medication, diagnose a patient's condition, or guarantee the actual volume delivered. Estimation accuracy depends on reliable drop detection, the selected drip factor, correct configuration, and the assumptions used by the software. The prototype is not clinically validated and must not be used to make patient-care decisions. Clinical deployment would require further engineering, testing, and regulatory assessment.

1.6 Expected Outcomes

The expected outcome of the project is a working academic prototype that demonstrates automated IV drip monitoring using embedded hardware and IoT-based software integration.

The expected outcomes include:

1. Drop detection and counting: The IR sensor and NodeMCU are expected to detect and count drops passing through the drip chamber under suitable test conditions.
2. Parameter estimation: The software is expected to calculate drip rate and estimate delivered volume, remaining fluid volume, and remaining infusion time from the detected drops and configured values.
3. Status indication: The programmed logic is expected to classify readings into normal, attention, and critical conditions and operate the corresponding LEDs and buzzer.
4. Wireless data transfer: The NodeMCU is expected to communicate readings to the Python-based dashboard over a compatible Wi-Fi network.
5. Dashboard and data storage: The dashboard is expected to display monitoring information, while SQLite is expected to store readings for subsequent review.
6. Functional demonstration: Controlled tests are expected to help identify drop-detection limitations, communication issues, and areas for improvement.

These outcomes describe the intended performance of the proposed system; they are not claims of measured accuracy or proven clinical effectiveness. Actual performance must be established through testing and documented evaluation.

CHAPTER 2: LITERATURE REVIEW

2.1 Introduction

The development of automated IV infusion monitoring systems has attracted attention because conventional gravity-based infusion requires regular observation of drip flow and fluid availability. Research has explored optical sensors, infrared sensors, load cells, capacitive sensing, microcontrollers, wireless communication, and web-based monitoring platforms to automate different parts of this process.

Sardana, Kalra, and Sardana presented an internet-connected IV drip monitoring device that combines optical drop detection with capacitive fluid-level sensing and wireless data reporting [1]. Venkatesh et al. developed DripOMeter, an open-source opto-electronic system that detects drops, estimates infused volume, and generates alerts based on configured conditions [2]. Other studies have investigated wireless drip-rate monitoring, fluid-level alerts, bubble detection, and IoT-connected infusion monitoring [3]–[7].

These studies provide a technical foundation for the proposed DripGuard system. DripGuard focuses on detecting drops with an IR sensor, processing readings using a NodeMCU ESP8266, estimating infusion parameters, and transmitting data to a Python dashboard backed by an SQLite database. Its contribution is the integration of these functions into an accessible academic prototype. The literature also indicates that estimated volume and infusion status depend on sensor reliability, calibration, and correct configuration; therefore, practical testing is necessary before drawing conclusions about performance.

2.2 Literature Review Table

Paper	Author	Year	Method	Findings	Research Gap
[1]	P. Sardana, M. Kalra, and A. Sardana	2019	Optical drop detection, capacitive level sensing, microcontroller, GSM and cloud communication.	Presents a retrofit IV monitoring platform with wireless reporting and testing under different operating conditions.	The proposed DripGuard uses a simpler IR-sensor and Wi-Fi arrangement; a comparative evaluation is needed to determine its detection reliability and limitations.
[3]	P. K. Patnaik et al.	2019	LED–LDR sensing, microcontroller, and Zigbee wireless communication.	Monitors drip rate, displays it locally, and can notify a central station when the rate moves outside a configured range.	DripGuard instead proposes a Python dashboard, HTTP-based data exchange, and SQLite storage; these features need functional testing.

Design and Development of IoT-Enabled IV Infusion Rate Monitoring and Control Device [4]	N/A	2020	IR-based drop detection, Wi-Fi-enabled controller, actuator, server, and web/mobile interfaces.	Describes connected monitoring with a control mechanism and online data visualization.	DripGuard is intended for monitoring and estimation only, without automatic physical control of the IV flow.
[5]	H. Chauhan et al.	2021	IoT-based monitoring of IV fluid using sensors and a microcontroller.	Explores connected fluid monitoring intended to reduce dependence on repeated manual observation.	DripGuard concentrates on drop counting and estimated infusion parameters; the estimates must be compared with measured test values.
[6]	J. K. Dhanoa et al.	2022	Light-dependent resistor-based sensing for fluid level and air-bubble-related alerts, with IoT notifications.	Describes visual and audible alerts and notification when a critical fluid level or bubble condition is detected.	The current DripGuard scope does not include dedicated bubble detection or a separate fluid-level sensor.
DripOMeter [2]	K. Venkatesh et al.	2022	Open-source opto-electronic drop detection and infusion monitoring.	Tracks drops, estimates infused volume, and alerts users about deviations from a set drip rate or a target volume.	DripGuard needs its own controlled tests for drop-count accuracy, volume estimation, and alert behavior; performance cannot be assumed equivalent.
[7]	Nandhini M. et al.	2024	Load-cell-based fluid-weight estimation with GSM alerts.	Describes monitoring fluid availability and notifying staff when the fluid amount reaches a minimum level.	DripGuard uses drop counting rather than load-cell measurement, so its estimated remaining volume may be

					affected by missed or falsely detected drops.
IVigilance [8]	Harshal Patil et al.	2024	IoT-based sensing and microcontroller processing for IV monitoring and control.	Describes collection of drip-related data and connected monitoring, with automated control features.	DripGuard prioritizes a lower-complexity monitoring prototype and database-backed dashboard rather than automated infusion control.
[9]	Teguh Afrianda et al.	2024	IoT-based IV monitoring using a wireless sensor network and optocoupler-based sensing.	Investigates wireless communication for monitoring IV infusion conditions.	DripGuard proposes a direct NodeMCU-to-Python communication pathway; its stability under network interruption must be tested.
IoT Enabled Automated IV Fluid Control and Cardiovascular Health Monitoring System [10]	N/A	2025	Connected infusion monitoring combined with automated flow control and cardiovascular monitoring.	Describes an integrated approach to IV monitoring, alerts, and additional patient-related measurements.	These additional functions are outside DripGuard's current scope; its main focus remains drop counting, estimation, alerts, and dashboard logging.
Real-Time Intravenous Infusion Monitoring using a Multi-Sensor IoT Framework [11]	N/A	2026	ESP32 with IR-based drop detection, ultrasonic fluid-level sensing, temperature sensing, and a web interface.	Describes multi-sensor monitoring with remote display and alerts.	DripGuard uses a simpler sensor configuration; comparative testing would be needed to assess the trade-off between simplicity and the information available from multiple sensors.

2.3 Research Gap

The reviewed studies demonstrate several approaches to IV infusion monitoring, including optical drop detection, fluid-level sensing, load-cell measurement, wireless communication, and automated

alerts. However, the selected approaches differ in hardware complexity, sensing methods, communication architecture, and monitoring features.

Based on this review, the following project-specific gaps and development needs have been identified:
1. Integrated monitoring and estimation: DripGuard aims to combine drop counting, drip-rate calculation, estimated delivered volume, remaining volume, and estimated remaining time in one prototype.
2. Accessible hardware and software: The system uses a NodeMCU ESP8266, an IR sensor, a Python dashboard, and SQLite to demonstrate an integrated monitoring workflow using commonly available components.
3. Local and remote alerts: The prototype combines LED indicators and a buzzer with dashboard-based status monitoring.
4. Data storage and review: The use of an SQLite database provides a way to store and inspect monitoring records instead of relying only on live readings.
5. Need for controlled evaluation: Drop-detection errors, sensor positioning, changing ambient conditions, network interruptions, and the assumptions used in volume estimation can affect system behavior. These factors must be evaluated through repeatable tests.

The proposed work addresses these needs by integrating sensing, processing, alert generation, wireless communication, and data logging in a single academic prototype. These are project-specific development goals rather than claims that earlier systems lack all such features. The prototype's reliability and usefulness must be established through testing, and the system is not intended for clinical use.

2.4 Novelty / Innovation

The proposed DripGuard system focuses on integrating low-cost embedded hardware with a software-based monitoring and data-recording platform. Its main contribution is the implementation of a complete academic prototype that connects IV drop detection, parameter estimation, status indication, wireless data exchange, and dashboard visualization.

The key features of the proposed system are:
1. IR-based drop detection: An IR sensor detects drops passing through the IV drip chamber, and the NodeMCU ESP8266 processes the sensor output.
2. Infusion parameter estimation: Drop counts are used to estimate the drip rate, delivered volume, remaining fluid volume, and remaining infusion time based on configured parameters.
3. Three-level status indication: Green, yellow, and red LEDs indicate the programmed normal, attention, and critical conditions, while a buzzer provides an audible alert under the configured critical condition.
4. Two-way communication: The NodeMCU sends readings to the Python backend, while the backend can send configurable parameters to the device.
5. Dashboard and database integration: The Python dashboard presents monitoring information, and SQLite stores readings for later review.
6. Multi-bed demonstration: The dashboard can represent one physical monitoring unit and additional simulated beds, demonstrating the software layout for a larger monitoring environment.

The innovation of DripGuard lies primarily in the integration and demonstration of these features in the project's chosen hardware and software architecture, rather than in claiming a new sensing principle. Its effectiveness will depend on successful implementation and controlled evaluation of drop detection, calculations, alerts, communication, and data storage.

References for Chapter 2

These references correspond to the numbered studies in the literature review table. Verify the final formatting and complete publication details against the original papers before submitting the full synopsis.

[1] P. Sardana, M. Kalra, and A. Sardana, "Design, Fabrication, and Testing of an Internet Connected Intravenous Drip Monitoring Device," *Journal of Sensor and Actuator Networks*, vol. 8, no. 1, Art. no. 2, 2019, doi: 10.3390/jsan8010002.

[2] K. Venkatesh, S. S. Alagundagi, V. Garg, K. Pasala, D. Karia, and M. Arora, "DripOMeter: An open-source opto-electronic system for intravenous (IV) infusion monitoring," *HardwareX*, vol. 12, Art. no. e00345, 2022, doi: 10.1016/j.ohx.2022.e00345.

[3] P. K. Patnaik, S. K. Nayak, A. Dey, S. S. Ray, and K. Pal, "Development of a wireless intravenous drip rate monitoring device," *International Journal of Sensor Networks*, vol. 29, no. 3, pp. 159–170, 2019, doi: 10.1504/IJSNET.2019.098287.

[4] "Design and Development of IOT enabled IV infusion rate monitoring and control device for precision care and portability," in *Proceedings of the 4th International Conference on Electronics, Communication and Aerospace Technology (ICECA)*, 2020, doi: 10.1109/ICECA49313.2020.9297376.

[5] H. Chauhan, V. Verma, D. Gupta, and S. Gupta, "IoT-based automatic intravenous fluid monitoring system for smart medical environment," *International Journal of Computer Applications in Technology*, vol. 66, no. 2, pp. 154–164, 2021, doi: 10.1504/IJCAT.2021.119767.

[6] J. K. Dhanoa *et al*., "Smart Intravenous Drip Monitoring System with Bubble Detection Indicator using IoT," in *Proceedings of the 2nd International Conference on Intelligent Technologies (CONIT)*, 2022, doi: 10.1109/CONIT55038.2022.9848118.

[7] M. Nandhini, S. Sambooranalaxmi, K. Valarmathi, P. A. Mathina, R. Ramalakshmi, and M. Mahalakshmi, "Smart Intravenous Drip Monitoring system based on IoT for Advancements in Healthcare," 2024, doi: 10.4108/eai.23-2-2024.2347041.

[8] H. Patil, M. Ram, V. Gharat, and S. Kudoo, "IVigilance - IOT based Intravenous drip monitoring and controlling system," *International Journal of Creative Research Thoughts*, vol. 12, no. 4, pp. b925–b934, 2024.

[9] T. Afrianda, Y. Yulkifli, Y. Yohandri, and A. Asman, "Design and Implementation of an IoT-Based Intravenous Infusion Monitoring System Using Wireless Sensor Network," *Applied Engineering, Innovation, and Technology*, vol. 1, no. 2, 2024.

[10] R. K. Rajeswari, G. Gayathri, K. Kiruthika, and S. Saranya, "IOT Enabled Automated IV Fluid Control and Cardiovascular Health Monitoring System," *International Journal for Multidisciplinary Research*, vol. 7, no. 5, 2025, doi: 10.36948/ijfmr.2025.v07i05.55791.

[11] "Real-Time Intravenous Infusion Monitoring using a Multi-Sensor IoT Framework," in *Proceedings of the 8th International Conference on Inventive Material Science and Applications (ICIMA)*, 2026, doi: 10.1109/ICIMA68728.2026.11564647.

CHAPTER 3: PROPOSED METHODOLOGY

3.1 Proposed System

DripGuard is an IoT-based IV infusion monitoring prototype that combines sensor interfacing, embedded processing, wireless communication, and software-based visualization. The system uses an IR sensor to detect drops passing through an IV drip chamber. A NodeMCU ESP8266 receives the sensor output, counts detected drops, and calculates infusion-related parameters using predefined or configurable values.

The calculated information includes the drip rate, estimated delivered volume, estimated remaining fluid volume, and estimated remaining infusion time. Based on the programmed decision rules, the system classifies the infusion status as normal, attention, or critical. A green LED represents normal status, a yellow LED represents attention status, and a red LED represents critical status. A buzzer provides an audible alert under the configured critical condition.

The NodeMCU communicates with a Python-based backend over Wi-Fi using HTTP requests. The backend can provide configuration parameters to the hardware and receive monitoring readings. A dashboard displays the information, while an SQLite database stores readings for later inspection.

The proposed system is designed for academic demonstration of sensor-based monitoring and IoT integration. It does not automatically control the IV flow and is not intended to replace medical supervision or certified medical equipment.

3.2 Working Principle

The working principle of DripGuard is based on detecting individual drops using an IR sensor and processing the detected events through the NodeMCU ESP8266.

1. Drop detection: The IR sensor is positioned near the transparent IV drip chamber so that the passage of a drop changes the sensor output.
2. Signal processing: The NodeMCU reads the sensor signal and counts the detected drops. Sensor positioning and sensitivity must be adjusted to reduce missed detections and false triggers.
3. Drip-rate calculation: The number of detected drops within a 60-second window is used to calculate the drip rate in drops per minute.
4. Fluid-volume estimation: The accumulated drop count and configured drip factor are used to estimate the delivered fluid volume.
5. Remaining-volume estimation: The estimated delivered volume is subtracted from the configured initial bag volume. Negative remaining-volume estimates are limited to zero.
6. Remaining-time estimation: When the calculated flow rate is greater than zero, the system estimates the remaining infusion time by dividing the estimated remaining volume by the calculated flow rate. If the flow rate is zero, a finite remaining-time estimate is not calculated.
7. Status classification: If no drops are detected during the measurement window, the status is classified as critical. Otherwise, if the drip rate is at least 80% of the configured target rate, the status is normal; a lower non-zero rate is classified as attention.
8. Alert generation: The green, yellow, or red LED indicates the corresponding status. The buzzer is activated for the programmed critical condition.

9. Wireless monitoring: The NodeMCU exchanges readings and configuration parameters with the Python backend over Wi-Fi. The dashboard presents the data, and SQLite stores monitoring records.

The calculations depend on the assumption that the configured drip factor correctly represents the administration set and that detected drops are counted reliably. The estimated values may differ from actual infused volume or remaining infusion time and must not be treated as clinical measurements.

Calculation formulas

Drip rate: Drip Rate = Window duration in minutes / Drops detected in the window

Estimated flow rate: Flow Rate (mL/min) = Drip Factor (drops/mL) / Drip Rate (drops/min)

Estimated delivered volume: Delivered Volume (mL) = Drip Factor (drops/mL) / Total counted drops

Estimated remaining volume: Remaining Volume = max(0,Bag Volume − Delivered Volume)

Estimated remaining time: Remaining Time (min) = Flow Rate (mL/min) / Remaining Volume (mL)
The remaining-time formula is applied only when the estimated flow rate is greater than zero.

3.3 System Architecture / Block Diagram

The system architecture shows the relationship between the sensing hardware, embedded controller, local alerts, and software monitoring platform.



Figure 3.1: Proposed system architecture of DripGuard.

The IR sensor provides the input signal for drop detection. The NodeMCU ESP8266 processes the signal, calculates the required parameters, and controls the local LED indicators and buzzer. The controller uses Wi-Fi to communicate with the Python backend through HTTP requests. The backend receives monitoring data, makes the information available to the dashboard, and stores readings in the SQLite database. Configuration values can also be transmitted from the backend to the controller. This two-way communication allows the system to use configurable monitoring parameters while presenting the resulting readings in a centralized interface.

3.4 Flow Chart

The following flowchart represents the main sequence of operations performed by the proposed system.

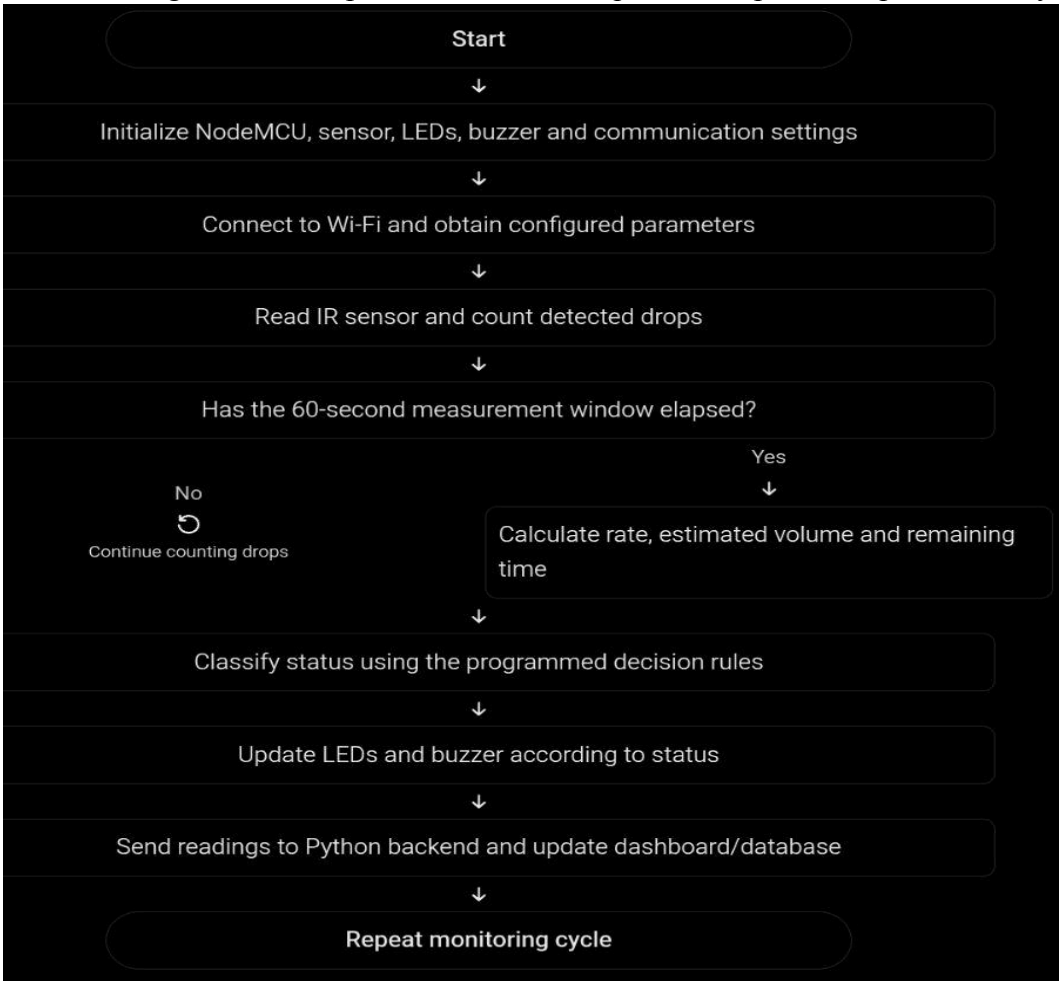


Figure 3.2: Flowchart of the DripGuard monitoring process.

After initialization, the system attempts to establish Wi-Fi communication and obtain the configured monitoring parameters. The NodeMCU continuously reads the IR sensor and counts detected drops. When the 60-second measurement window is complete, the controller calculates the drip rate and estimated infusion parameters. It then determines the status according to the programmed conditions, updates the local indicators, and transmits the readings to the Python backend. The process repeats to provide ongoing monitoring. If communication is unavailable, the dashboard may not receive updated data; therefore, communication status and recovery behavior should be considered during testing.

3.5 Methodology

The development of DripGuard follows a step-by-step methodology involving hardware assembly, sensor interfacing, firmware development, software integration, and functional testing.

Step 1: Hardware setup: The NodeMCU ESP8266, IR sensor, three LEDs, current-limiting resistors, and buzzer are connected according to the circuit design. The IR sensor is positioned near the IV drip chamber to detect passing drops.

Step 2: Sensor interfacing: The IR sensor output is connected to the selected digital input pin of the NodeMCU. Sensor positioning and sensitivity are adjusted to obtain a distinguishable output when drops pass through the detection area.

Step 3: Firmware development: The microcontroller firmware reads the sensor signal, counts drops, calculates the drip rate over a 60-second window, and estimates delivered volume, remaining volume, and remaining infusion time using the configured parameters.

Step 4: Status and alert logic: The firmware classifies the detected condition using the programmed thresholds. The LEDs indicate normal, attention, or critical status, and the buzzer is activated for the configured critical condition.

Step 5: Wi-Fi communication: The NodeMCU connects to a compatible 2.4 GHz Wi-Fi network and exchanges monitoring readings and configuration parameters with the Python backend through HTTP requests.

Step 6: Dashboard and database development: The Python application receives readings, presents them on the dashboard, and stores monitoring records in an SQLite database. The database can be inspected using DB Browser for SQLite.

Step 7: System integration and testing: The hardware and software are integrated and tested under controlled demonstration conditions. Tests should check drop detection, calculation consistency, status transitions, buzzer and LED operation, data transfer, dashboard updates, and database recording.

Step 8: Evaluation and documentation: The observed behavior is compared with expected behavior, and any missed detections, false triggers, communication errors, or calculation limitations are documented. Performance claims should be made only after collecting and analyzing test data.

The methodology is intended to demonstrate the feasibility of the proposed monitoring prototype. It does not establish clinical accuracy or suitability for patient care.

3.6 Components / Materials

Component	Specification	Quantity
NodeMCU ESP8266	Processes sensor signals, calculates parameters, controls alerts, and communicates over Wi-Fi.	1
IR sensor module	Detects drops passing through the IV drip chamber.	1

Active buzzer	Provides an audible alert during the programmed critical condition.	1
Green LED	Indicates normal status.	1
Yellow LED	Indicates attention status.	1
Red LED	Indicates critical status.	1
220 Ω resistors	Limit current through the LEDs.	3
Breadboard	Supports temporary circuit assembly.	1
Jumper wires	Connect the controller, sensor, LEDs, and buzzer.	F-F: 10, M-M: 10, M-F: 10
IV administration set	Provides the drip chamber, tubing, spike, and roller clamp for the demonstration setup.	1
USB data cable	Powers and programs the NodeMCU.	1
Computer or laptop	Runs the Python backend and dashboard and provides access to the SQLite database.	1
Python and required libraries	Implement the backend, communication, and dashboard functions.	1 (Software)
SQLite	Stores monitoring readings.	1 (Software)
Arduino IDE or compatible development environment	Used to develop and upload firmware to the NodeMCU.	1 (Software)
Compatible 2.4 GHz Wi-Fi network	Enables communication between the NodeMCU and Python backend.	1

Table 3.1: Hardware and software components of DripGuard.

Important implementation note: Verify the sensor module's output polarity and the buzzer's operating voltage before powering the circuit. The NodeMCU uses 3.3 V logic, and its GPIO pins must not be exposed to unsafe voltage levels. The red LED connection to D3 (GPIO0) also requires care because this pin affects the controller's boot mode; if it causes boot problems, use a suitable alternative pin and update the firmware accordingly.

CHAPTER 4: PROJECT PLANNING

4.1 Work Breakdown Structure

The Work Breakdown Structure (WBS) divides the DripGuard project into manageable tasks to support systematic development, integration, testing, and documentation.
The project is divided into the following major activities:

1. Requirement Analysis: Study the project objectives, identify functional requirements, select suitable hardware and software, and define the monitoring parameters.
2. Literature Review and System Design: Review related IV monitoring systems and prepare the system architecture, circuit connections, flowchart, and software design.
3. Hardware Assembly: Assemble the NodeMCU ESP8266, IR sensor, LEDs, buzzer, resistors, and supporting connections.
4. Firmware Development: Develop and upload the controller program for drop detection, drop counting, parameter estimation, status classification, and alert control.
5. Dashboard and Database Development: Develop the Python backend and dashboard, implement communication with the controller, and configure SQLite data storage.
6. System Integration: Connect the hardware and software, configure Wi-Fi communication, and verify the exchange of readings and parameters.
7. Testing and Evaluation: Test sensor response, parameter calculations, LED and buzzer operation, dashboard updates, and database recording under controlled conditions.
8. Documentation and Final Demonstration: Prepare the synopsis, circuit and software documentation, testing records, presentation, and project demonstration.

This task-based structure supports progress tracking and helps identify integration issues before the final demonstration.

4.2 Gantt Chart

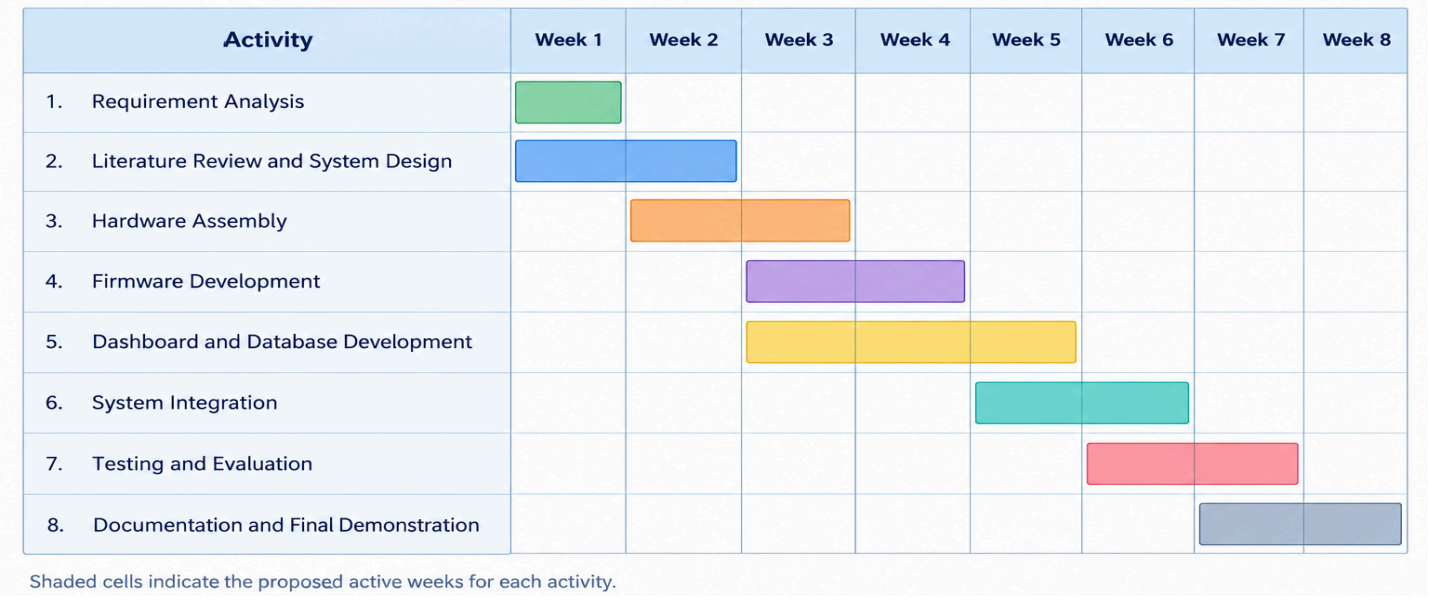


Figure 4.1: Proposed Gantt chart for DripGuard project activities.

The project schedule begins with requirement analysis and literature review, followed by hardware and software development. Hardware assembly and firmware development are planned alongside dashboard development to allow early identification of integration issues. System integration is followed by functional testing and evaluation. The final weeks are allocated to completing documentation, preparing the demonstration, and addressing issues identified during testing.
The schedule is indicative and may be modified according to component availability, implementation progress, academic deadlines, and the time required to resolve hardware or software issues.

4.3 Roles and Responsibilities

The following table provides a suggested division of responsibilities for a student project team.

No.	Team Role	Responsibilities
1	Project Coordinator	Coordinates project activities, monitors progress, communicates with the project guide, and ensures that milestones are followed.
2	Hardware and Sensor Developer	Assembles the circuit, interfaces the IR sensor, connects the LEDs and buzzer, and checks the electrical connections.
3	Firmware Developer	Implements drop counting, parameter calculations, status classification, and local alert control on the NodeMCU ESP8266.
4	Software and Dashboard Developer	Develops the Python backend and dashboard, implements HTTP communication, and manages SQLite data storage.
5	Testing and Documentation Coordinator	Plans functional tests, records observations, documents limitations, prepares project diagrams, and supports the final report and presentation.

In a small team, one member may perform multiple roles, and all members should contribute to integration, testing, and project documentation. The final allocation must reflect the actual work performed by each member.

4.4 Risk Assessment

Risk assessment identifies possible problems that could affect the development, operation, or demonstration of DripGuard.

The following table lists anticipated risks and proposed mitigation measures.

Risk	Impact	Mitigation
Missed or false drop	Incorrect drip rate	Adjust sensor alignment and sensitivity, use repeatable

detection	and volume estimates.	test conditions, and compare counts with manually observed drops.
Ambient light or sensor interference	Unstable sensor output.	Shield the sensing area where appropriate, secure the sensor position, and test under different lighting conditions.
Incorrect wiring or voltage levels	Component malfunction or controller damage.	Verify the circuit before powering it, use appropriate LED resistors, and ensure that GPIO voltage limits are respected.
NodeMCU boot failure	The prototype may fail to start.	Check boot-sensitive pins, particularly GPIO0/D3, and revise the pin assignment if necessary.
Wi-Fi or HTTP communication failure	The dashboard may display stale or missing readings.	Verify network settings, implement communication timeouts and reconnection behavior where possible, and indicate unavailable data.
Incorrect configuration values	Misleading flow, volume, or time estimates.	Verify the bag volume, drip factor, target drip rate, and units before each test.
Dashboard or database error	Readings may not display or be stored correctly.	Validate incoming data, handle database errors, and check saved records using DB Browser for SQLite.
Component availability or damage	Delays in assembly and testing.	Check components early, keep suitable spare wires and basic components, and maintain a backup of firmware and software.
Limited testing time	Some faults may remain undetected.	Prioritize core functions, follow the project schedule, and document tests that could not be completed.
Misinterpretation as a clinically approved device	Unsafe reliance on unvalidated readings.	Clearly label the system as an academic prototype and do not use it to guide patient care.

Risk assessment should be revisited during development. Any observed failure should be recorded with its cause, corrective action, and retest outcome. The proposed mitigations reduce development and demonstration risks but do not establish clinical safety.

CHAPTER 5: RESOURCES & FEASIBILITY

5.1 Technical Feasibility

DripGuard is technically feasible as an academic prototype because it uses commonly available electronic components and established software technologies. The NodeMCU ESP8266 provides digital input/output and built-in Wi-Fi connectivity, enabling it to interface with an IR sensor, control status indicators, and exchange data with a computer-based application.
The IR sensor is used to detect drops passing through the IV drip chamber. The microcontroller processes the sensor readings and calculates the drip rate and estimated infusion parameters according to the programmed logic. A Python-based backend receives the readings, displays them on a dashboard, and stores the data in an SQLite database.

The proposed system can be developed using an Arduino-compatible development environment, Python, and SQLite. These technologies are suitable for a student project involving embedded programming, sensor interfacing, wireless communication, and basic data management.

The main technical challenges include accurate drop detection, appropriate sensor positioning, reliable Wi-Fi communication, correct parameter configuration, and consistent data recording. These challenges can be addressed through circuit verification, software debugging, and controlled testing. Technical feasibility for demonstration does not establish clinical accuracy or medical-device suitability.

5.2 Economic Feasibility

The project is economically feasible for an academic prototype because it relies on relatively low-cost electronic components and open-source software. The NodeMCU ESP8266 combines microcontroller functionality and Wi-Fi connectivity in one board, reducing the need for a separate wireless communication module.

The IR sensor, LEDs, resistors, buzzer, breadboard, and jumper wires are commonly available components that can be used to assemble the prototype. Python and SQLite can be used without purchasing commercial software licences for the proposed basic implementation.

The main expenses are expected to involve the controller, sensor, circuit components, wiring, and IV administration set. Additional costs may arise if components need replacement or if the project requires extra hardware for testing. The estimated budget is presented in Section 5.4 and should be updated using actual supplier quotations.

Overall, the proposed system provides a practical way to demonstrate embedded sensing and IoT-based monitoring within a student-project budget. The estimate covers prototype development only and excludes clinical certification, regulatory approval, and large-scale deployment.

5.3 Operational Feasibility

DripGuard is operationally feasible for an academic demonstration because its hardware and software functions can be operated using a NodeMCU, an IR sensor, a compatible Wi-Fi network, and a computer running the Python application.

During operation, the sensor detects drops, the microcontroller processes the readings, and the LEDs and buzzer indicate the programmed status. The Python dashboard displays the received information, while the SQLite database stores monitoring records for review.

The system requires correct sensor alignment, suitable electrical connections, a compatible 2.4 GHz Wi-Fi network, and correctly configured parameters. Users must also understand that the estimated volume and remaining time depend on drop-count accuracy and the configured drip factor.

The prototype can be used for controlled demonstrations and educational testing. It is not intended for actual patient monitoring or treatment decisions. Clinical use would require independent safety testing, performance validation, appropriate safeguards, and applicable regulatory approval.

5.4 Budget Estimation

Item	Quantity	Cost
NodeMCU ESP8266 development board	1	₹410
IR sensor module	1	₹30
Active buzzer	1	₹50
LEDs (green, yellow, red)	3	₹20
220 Ω resistors	3	₹10
Breadboard	1	₹180
Jumper wires	1 set	₹140
IV administration set	1	₹150
USB data cable	1	₹130
Miscellaneous and contingency	—	₹300
Estimated Total		₹1420

Table 5.1. The final cost may vary according to supplier, component quality, delivery charges, and local availability.

5.5 Bill of Materials (BOM)

No.	Component	Specification / Description	Quantity	Purpose
1	NodeMCU ESP8266	Wi-Fi-enabled development board with 3.3 V logic	1	Main controller and wireless communication

2	IR sensor module	Digital-output IR module; verify actual module specs	1	IV drop detection
3	Active buzzer	Compatible with selected supply voltage and drive circuit	1	Audible critical alert
4	Green LED	Standard indicator LED	1	Normal status indication
5	Yellow LED	Standard indicator LED	1	Attention status indication
6	Red LED	Standard indicator LED	1	Critical status indication
7	Resistor	220 Ω current-limiting resistor	3	LED current limitation
8	Breadboard	Solderless prototyping board	1	Circuit assembly
9	Jumper wires	Compatible male-male, male-female, or female-female wires	1 set	Electrical connections
10	IV administration set	Drip chamber, tubing, spike, and roller clamp	1	Demonstration of drop detection
11	USB data cable	Cable compatible with the NodeMCU connector	1	Programming and power
12	Computer or laptop	Capable of running the required Python environment	1	Backend, dashboard, and database
13	Software tools	Arduino IDE or compatible environment, Python, SQLite	As required	Firmware development and data management
14	Wi-Fi network	2.4 GHz wireless network	1	Communication between the NodeMCU and backend

The bill of materials identifies the components required to assemble the prototype. Unlike the budget table, the BOM focuses on component specifications, quantities, and intended use.Note: The computer, Wi-Fi network, and software are included as operational requirements and may not incur additional component costs if already available. Confirm the actual IR sensor module and buzzer specifications before assembly.

CHAPTER 6: EXPECTED RESULTS

6.1 Expected Results

The proposed DripGuard system is expected to demonstrate automated IV drop detection and basic infusion monitoring using an IR sensor, NodeMCU ESP8266, and a Python-based dashboard. The prototype is intended to combine hardware-based sensing with software-based parameter estimation, status indication, and data recording.

The expected results are as follows:

1. The IR sensor is expected to detect drops passing through the IV drip chamber under suitable testing conditions.
2. The NodeMCU is expected to count detected drops and calculate the drip rate over a defined measurement interval.
3. The system is expected to estimate delivered volume, remaining fluid volume, and remaining infusion time using the configured drip factor and bag volume.
4. The programmed status logic is expected to classify monitored conditions as normal, attention, or critical. 5. The LEDs and buzzer are expected to provide local visual and audible indications according to the detected status.
6. The Python dashboard is expected to display readings received from the controller over Wi-Fi.
7. The SQLite database is expected to store monitoring readings for later inspection.

These results will be evaluated through controlled functional tests. Actual detection accuracy, calculation error, communication reliability, and system response must be measured before any quantitative performance claims can be made.

6.2 Expected Outcomes and Performance

The project is expected to produce a working academic prototype that integrates drop detection, embedded processing, wireless communication, dashboard visualization, and database storage.

The following performance aspects are planned for evaluation:

1. Drop-count consistency: Compare sensor-detected drops with manually observed drops during controlled trials.
2. Calculation correctness: Verify the drip-rate, delivered-volume, remaining-volume, and remaining-time calculations using known test inputs.
3. Alert operation: Confirm that the LEDs and buzzer respond according to the programmed status rules.
4. Communication performance: Check whether the NodeMCU can transmit readings to the Python backend and receive configuration parameters over Wi-Fi.
5. Dashboard and database operation: Verify that readings appear correctly on the dashboard and are stored in SQLite.
6. Operational stability: Observe sensor behavior, communication interruptions, and software errors during repeated tests.

The outcome will be documented using test observations and, where appropriate, numerical measurements such as drop-count error and successful data-transfer rate. No accuracy percentage or response-time guarantee is claimed before testing.
The system's calculations represent estimates derived from drop counts and configured parameters. The results are intended to evaluate the prototype's functionality, not to establish medical-grade accuracy or clinical effectiveness.

6.3 Industrial Relevance

DripGuard is relevant to the broader development of low-cost IoT-based monitoring systems for healthcare-related applications. Its design demonstrates how sensors, microcontrollers, wireless communication, dashboards, and databases can be integrated to observe a physical process and present information digitally.

The project may be useful as an educational reference for students studying embedded systems, the Internet of Things, sensor interfacing, Python programming, and basic healthcare technology. The architecture may also serve as a starting point for future research into infusion monitoring, remote observation, and data logging.

However, the current prototype is not a certified medical device and has not been established as suitable for clinical deployment. Any industrial or hospital application would require appropriate engineering validation, risk management, electrical safety assessment, cybersecurity safeguards, and compliance with applicable medical-device regulations.

6.4 Sustainability and Environmental Impact

DripGuard has a limited but positive potential environmental contribution through its use of reusable electronic components and open-source software. The NodeMCU, sensor module, breadboard, LEDs, and buzzer can be reused during repeated demonstrations and software development, reducing the need to purchase new components for every test.

The system's digital dashboard and electronic data storage may reduce the need for handwritten records during prototype demonstrations. However, the actual reduction in paper use or environmental impact has not been measured.

The project also generates electronic waste if damaged components, cables, or circuit boards are discarded improperly. Components should therefore be reused where possible, and electronic waste should be handled through appropriate collection and recycling channels. The IV administration set should be used only in accordance with relevant hygiene and disposal requirements.

Overall, the project encourages resource-conscious prototyping, reuse of electronic components, and responsible management of electronic waste.

6.5 Sustainable Development Goals (SDG) Mapping

The project has potential links to the following United Nations Sustainable Development Goals (SDGs).

SDG 3 — Good Health and Well-Being DripGuard explores automated observation of IV drip activity and alerts. The prototype may support educational work on healthcare monitoring, but it has not been validated to improve patient outcomes.	
SDG 9 — Industry, Innovation and Infrastructure The project applies embedded electronics, sensor technology, IoT communication, and software integration to develop an accessible monitoring prototype.	
SDG 12 — Responsible Consumption and Production Reusable prototype components and responsible electronic-waste handling support resource efficiency and more responsible consumption practices.	

Figure 6.1: Sustainable Development Goals relevant to the DripGuard project.

DripGuard is primarily aligned with SDG 3 through its focus on healthcare-related monitoring research and SDG 9 through the integration of sensors, embedded hardware, and IoT technology. It also has a potential connection to SDG 12 because reusable electronic components can support resource-efficient prototyping. These connections represent the project's intended relevance; they do not establish a measured contribution to health outcomes, industrial development, or environmental improvement.

6.6 Future Scope

The current DripGuard prototype provides a foundation for further development in sensing, communication, data analysis, and system reliability. The following improvements may be explored in future versions:

1. Improved drop detection: Refine sensor positioning, signal filtering, and drop-counting logic to reduce missed detections and false triggers.
2. Additional sensors: Investigate independent fluid-level sensing or air-bubble detection, subject to appropriate sensor selection and validation.
3. Enhanced dashboard: Add historical graphs, configurable alerts, device connectivity indicators, and improved visualization of monitoring records.
4. Network reliability: Improve reconnection behavior, communication error handling, and the identification of stale or missing readings.
5. Multi-device integration: Extend the software architecture to support multiple physical monitoring units, each with its own controller and device identification.
6. Data analysis: Evaluate stored readings to identify patterns, detect inconsistent measurements, and compare estimates with reference measurements.
7. Power and enclosure improvements: Develop a more robust enclosure, safer wiring, and suitable power arrangements for demonstration purposes.
8. Formal validation: Conduct systematic accuracy, reliability, usability, and safety testing before considering any application beyond an academic prototype.

These developments could improve the system's functionality and provide opportunities for further research. Any future clinical application would require extensive validation and the necessary regulatory approvals.

CHAPTER 7: Conclusion

DripGuard: Smart IV Infusion Monitoring and Prediction System is a proposed IoT-based prototype designed to monitor IV drip activity, estimate infusion-related parameters, and provide local and remote status information. The system integrates an IR sensor with a NodeMCU ESP8266 to detect and count drops passing through an IV drip chamber. Based on the detected drops and configured parameters, it estimates the drip rate, delivered fluid volume, remaining fluid volume, and estimated remaining infusion time.

The proposed system also incorporates green, yellow, and red LED indicators and a buzzer to communicate the programmed monitoring status. Wi-Fi communication enables the NodeMCU to exchange readings and configuration parameters with a Python-based dashboard, while an SQLite database stores monitoring records for later inspection.

The project demonstrates the integration of embedded systems, sensor interfacing, wireless communication, software development, and data management in a low-cost academic prototype. Its development provides an opportunity to study automated drop detection and digital monitoring under controlled conditions. The reliability of the system will depend on sensor performance, correct configuration, communication stability, and the accuracy of its estimation methods.

In conclusion, DripGuard aims to provide a practical demonstration of IoT-based IV infusion monitoring and establish a foundation for future improvements in sensing, data analysis, and system reliability. The prototype is intended for educational and experimental purposes only. It is not a clinically validated medical device and must not replace trained healthcare supervision or be used to make patient-care decisions.

REFERENCES

[1] P. Sardana, M. Kalra, and A. Sardana, “Design, Fabrication, and Testing of an Internet Connected Intravenous Drip Monitoring Device,” *Journal of Sensor and Actuator Networks*, vol. 8, no. 1, Art. no. 2, 2019, doi: 10.3390/jsan8010002.
[2] K. Venkatesh, S. S. Alagundagi, V. Garg, K. Pasala, D. Karia, and M. Arora, “DripOMeter: An open-source opto-electronic system for intravenous (IV) infusion monitoring,” *HardwareX*, vol. 12, Art. no. e00345, 2022, doi: 10.1016/j.ohx.2022.e00345.
[3] T. Afrianda, Y. Yulkifli, Y. Yohandri, and A. Asman, “Design and Implementation of an IoT-Based Intravenous Infusion Monitoring System Using Wireless Sensor Network,” *Applied Engineering, Innovation, and Technology*, vol. 1, no. 2, pp. 59–67, 2024, doi: 10.62777/aeit.v1i2.25.
[4] J.-K. Lee, K.-C. Yoon, and K. G. Kim, “Design of a Remote Monitoring System Based on Optical Sensors to Prevent Medical Accidents during Fluid Treatment,” *Applied Sciences*, vol. 11, no. 21, Art. no. 10124, 2021, doi: 10.3390/app112110124.
[5] K. K. et al., “A Survey of Systems used in the Monitoring and Control of Intravenous Infusion,” *International Journal of Engineering and Technology*, vol. 11, no. 1, pp. 114–119, 2019, doi: 10.21817/ijet/2019/v11i1/191101054.
[6] A. Sujiwa and R. R. Dianto, “Infusion Monitoring System for Patients Based on the Internet of Things (IoT) with Android Notification System,” *BEST: Journal of Applied Electrical, Science, & Technology*, vol. 4, no. 2, pp. 41–46, 2022, doi: 10.36456/best.vol4.no2.6172.
[7] G. Priyandoko, “Rancang Bangun Sistem Portable Monitoring Infus Berbasis Internet of Things,” *Jambura Journal of Electrical and Electronics Engineering*, vol. 3, no. 2, pp. 56–61, 2021, doi: 10.37905/jjeee.v3i2.10508.
[8] P. Abril-Jiménez et al., “Evaluating IoT-Based Services to Support Patient Empowerment in Digital Home Hospitalization Services,” *Sensors*, vol. 23, no. 3, Art. no. 1744, 2023, doi: 10.3390/s23031744.
[9] Espressif Systems, “ESP8266EX Datasheet.” [Online]. Available: https://www.espressif.com/sites/default/files/documentation/0a-esp8266ex_datasheet_en.pdf
[10] Arduino, “Arduino Documentation: Software and Development Tools.” [Online]. Available: https://docs.arduino.cc/software/
[11] Python Software Foundation, “sqlite3 — DB-API 2.0 Interface for SQLite Databases.” [Online]. Available: https://docs.python.org/3/library/sqlite3.html
[12] SQLite, “SQLite Documentation.” [Online]. Available: https://www.sqlite.org/docs.html
[13] Python Software Foundation, “Python Documentation.” [Online]. Available: https://docs.python.org/3/
[14] Espressif Systems, “ESP8266 RTOS SDK Documentation.” [Online]. Available: https://docs.espressif.com/projects/esp8266-rtos-sdk/en/latest/
[15] Arduino, “Getting Started with Arduino IDE 2.” [Online]. Available: https://docs.arduino.cc/software/ide-v2/tutorials/getting-started/ide-v2-downloading-and-installing

APPENDICES

Circuit Diagram:

The DripGuard prototype consists of a NodeMCU ESP8266 microcontroller, an IR sensor module, three status LEDs, and an active buzzer. The IR sensor detects drip activity, and the microcontroller processes the sensor output to provide visual and audible alerts based on the programmed monitoring conditions.

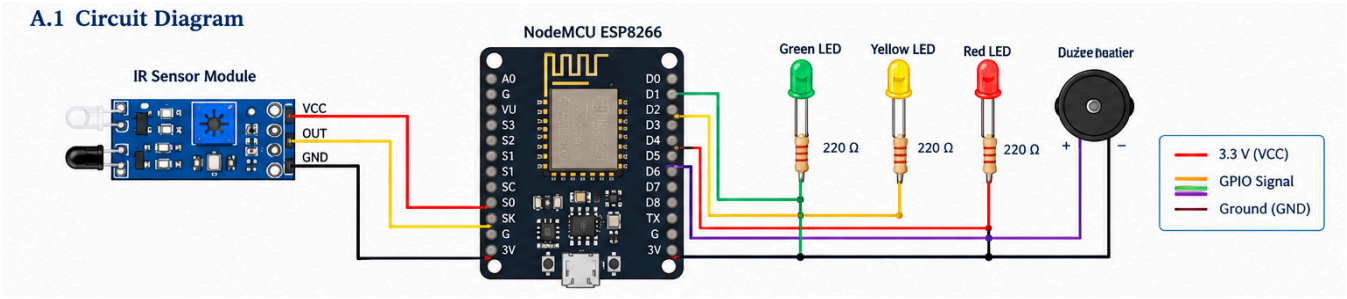


Figure A.1: Circuit Diagram of the DripGuard Prototype

Pin Connection Table:

No.	Component	Specification / Description	Quantity	Purpose
1	NodeMCU ESP8266	Wi-Fi-enabled development board with 3.3 V logic	1	Main controller and wireless communication
2	IR sensor module	Digital-output IR module; verify actual module specs	1	IV drop detection
3	Active buzzer	Compatible with selected supply voltage and drive circuit	1	Audible critical alert
4	Green LED	Standard indicator LED	1	Normal status indication
5	Yellow LED	Standard indicator LED	1	Attention status indication
6	Red LED	Standard indicator LED	1	Critical status indication
7	Resistor	220 Ω current-limiting resistor	3	LED current limitation

8	Breadboard	Solderless prototyping board	1	Circuit assembly
9	Jumper wires	Compatible male-male, male-female, or female-female wires	1 set	Electrical connections
10	IV administration set	Drip chamber, tubing, spike, and roller clamp	1	Demonstration of drop detection
11	USB data cable	Cable compatible with the NodeMCU connector	1	Programming and power
12	Computer or laptop	Capable of running the required Python environment	1	Backend, dashboard, and database
13	Software tools	Arduino IDE or compatible environment, Python, SQLite	As required	Firmware development and data management
14	Wi-Fi network	2.4 GHz wireless network	1	Communication between the NodeMCU and backend

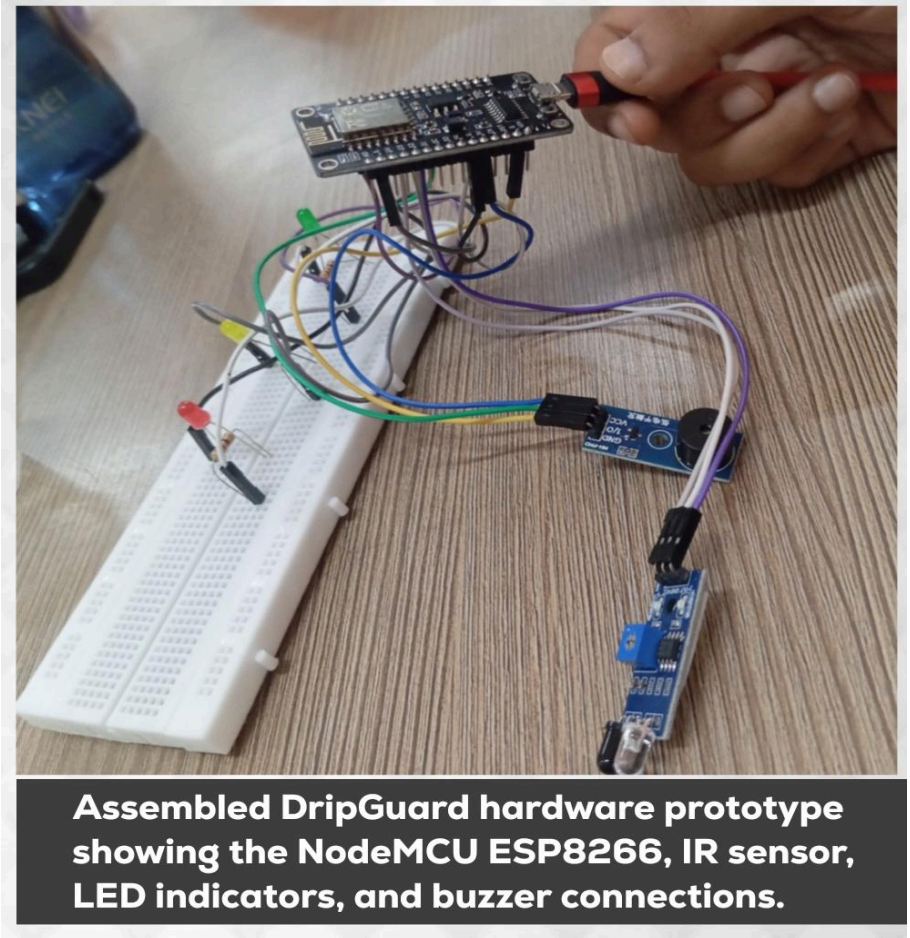


Figure A.2: Actual Hardware Prototype of Dripguard System

Working Principle:
The IR sensor detects drops in the IV drip chamber and sends signals to the NodeMCU ESP8266. The microcontroller counts the detected drops over a defined time interval and calculates the drip rate. Based on the configured drop factor and initial fluid volume, the system estimates the delivered volume, remaining volume, and approximate remaining infusion time.

The green LED indicates normal status, the yellow LED indicates an attention condition, and the red LED with the buzzer indicates a critical condition, according to the programmed logic. Monitoring information can also be transmitted through Wi-Fi to the Python-based dashboard for display and storage.

Note: The circuit must be verified against the actual hardware before implementation. GPIO0 (D3) is a boot-sensitive pin on the ESP8266 and may need to be changed if it interferes with startup. The prototype is intended for academic demonstration and is not a replacement for professional clinical monitoring.